

Milestone 3 Employee Attrition

May 26, 2026

```
[5]: import pandas as pd
url = "C:\\Users\\czarn\\Documents\\DSC550\\Employee Attrition.csv"
df = pd.read_csv(url)
```

```
[7]: df.head()
```

```
[7]:   Age  Attrition  BusinessTravel      Department  DistanceFromHome  \
0   37         Yes   Travel_Rarely  Research & Development             2
1   21         No   Travel_Rarely  Research & Development            15
2   45         No   Travel_Rarely  Research & Development             6
3   23         No   Travel_Rarely             Sales             2
4   22         No   Travel_Rarely  Research & Development            15

   Gender  JobInvolvement  JobLevel      JobRole  JobSatisfaction  \
0   Male                2          1  Laboratory Technician             3
1   Male                3          1   Research Scientist             4
2   Male                3          3   Research Director             1
3   Male                3          1  Sales Representative             1
4  Female                3          1  Laboratory Technician             4

   ... Higher_Education  Date_of_Hire  Date_of_termination  \
0   ...      Graduation    21-01-2021                 NaN
1   ...      Graduation    13-03-2021                 NaN
2   ...  Post-Graduation    23-01-2021                 NaN
3   ...              PHD    25-04-2021                 NaN
4   ...              PHD    14-06-2021                 NaN

   Status_of_leaving  Mode_of_work  Leaves  Absenteeism  Work_accident  \
0           Salary      OFFICE      4          2          No
1   Work Accident      WFH      5          2          No
2           Dept.Head      WFH      1          3          No
3   Work Accident      OFFICE      1          0          Yes
4  Better Opportunity      WFH      5          2          No

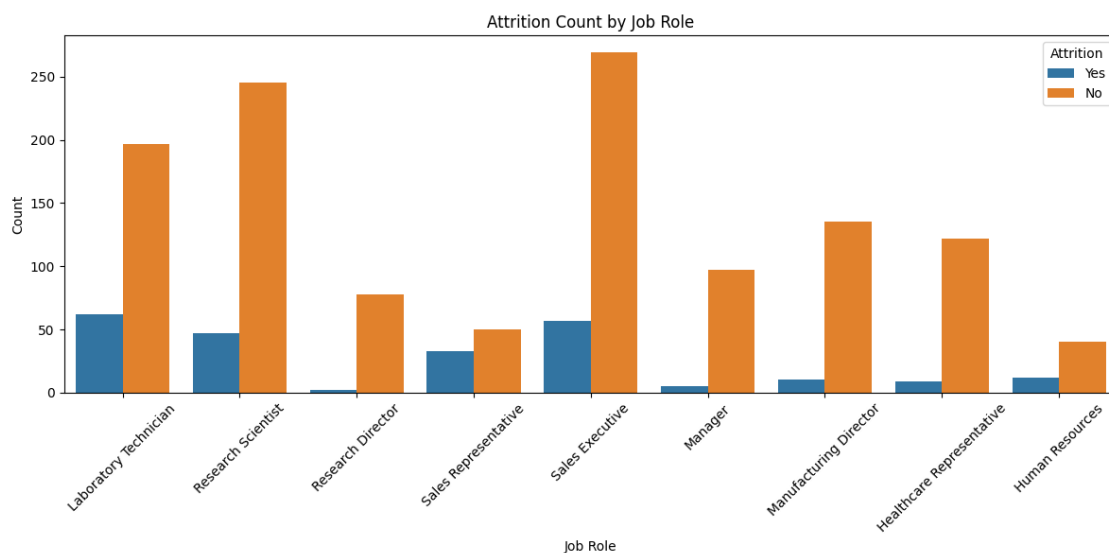
   Source_of_Hire  Job_mode
0      Job Event  Contract
1      Recruiter  Part Time
2      Job Event  Contract
```

```
3      Recruiter    FullTime
4      Job Event   Contract
```

[5 rows x 32 columns]

```
[9]: # Attrition Count by job role and mode of work
import seaborn as sns
import matplotlib.pyplot as plt

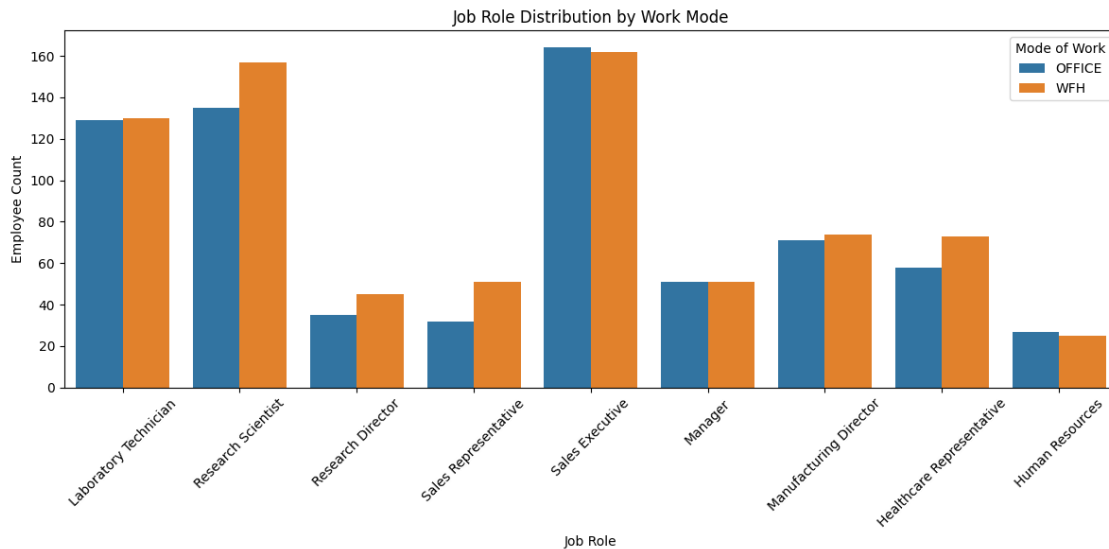
plt.figure(figsize=(12,6))
sns.countplot(data=df, x='JobRole', hue='Attrition')
plt.title('Attrition Count by Job Role')
plt.xticks(rotation=45)
plt.xlabel('Job Role')
plt.ylabel('Count')
plt.legend(title='Attrition')
plt.tight_layout()
plt.show()
```



Summary: Attrition seems more common in roles like Sales or technical roles like Research Scientist indicating potential stress or dissatisfaction.

```
[11]: # Breakdown by Work Mode
plt.figure(figsize=(12,6))
sns.countplot(data=df, x='JobRole', hue='Mode_of_work')
plt.title('Job Role Distribution by Work Mode')
plt.xticks(rotation=45)
plt.xlabel('Job Role')
plt.ylabel('Employee Count')
```

```
plt.legend(title='Mode of Work')
plt.tight_layout()
plt.show()
```



Summary: Most roles have similar distributions between office and remote work.

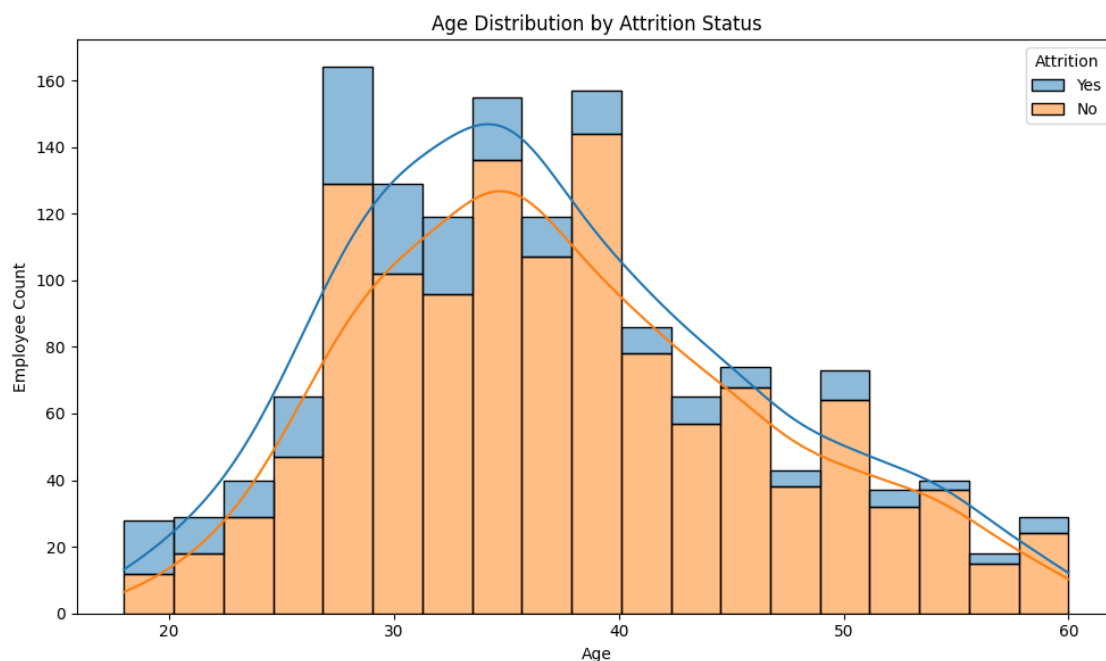
```
[15]: # Age Distribution by Attrition and Mode of Work
plt.figure(figsize=(10,6))
sns.histplot(data=df, x='Age', hue='Attrition', kde=True, multiple='stack')
plt.title('Age Distribution by Attrition Status')
plt.xlabel('Age')
plt.ylabel('Employee Count')
plt.tight_layout()
plt.show()
```

```
C:\Users\czarn\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a
future version. Convert inf values to NaN before operating instead.
    with pd.option_context('mode.use_inf_as_na', True):
C:\Users\czarn\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass a
length-1 tuple to get_group in a future version of pandas. Pass `(name,)`
instead of `name` to silence this warning.
    data_subset = grouped_data.get_group(pd_key)
C:\Users\czarn\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass a
length-1 tuple to get_group in a future version of pandas. Pass `(name,)`
instead of `name` to silence this warning.
    data_subset = grouped_data.get_group(pd_key)
```

```

C:\Users\czarn\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass a
length-1 tuple to get_group in a future version of pandas. Pass `(name,)`
instead of `name` to silence this warning.
    data_subset = grouped_data.get_group(pd_key)
C:\Users\czarn\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass a
length-1 tuple to get_group in a future version of pandas. Pass `(name,)`
instead of `name` to silence this warning.
    data_subset = grouped_data.get_group(pd_key)
C:\Users\czarn\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass a
length-1 tuple to get_group in a future version of pandas. Pass `(name,)`
instead of `name` to silence this warning.
    data_subset = grouped_data.get_group(pd_key)

```



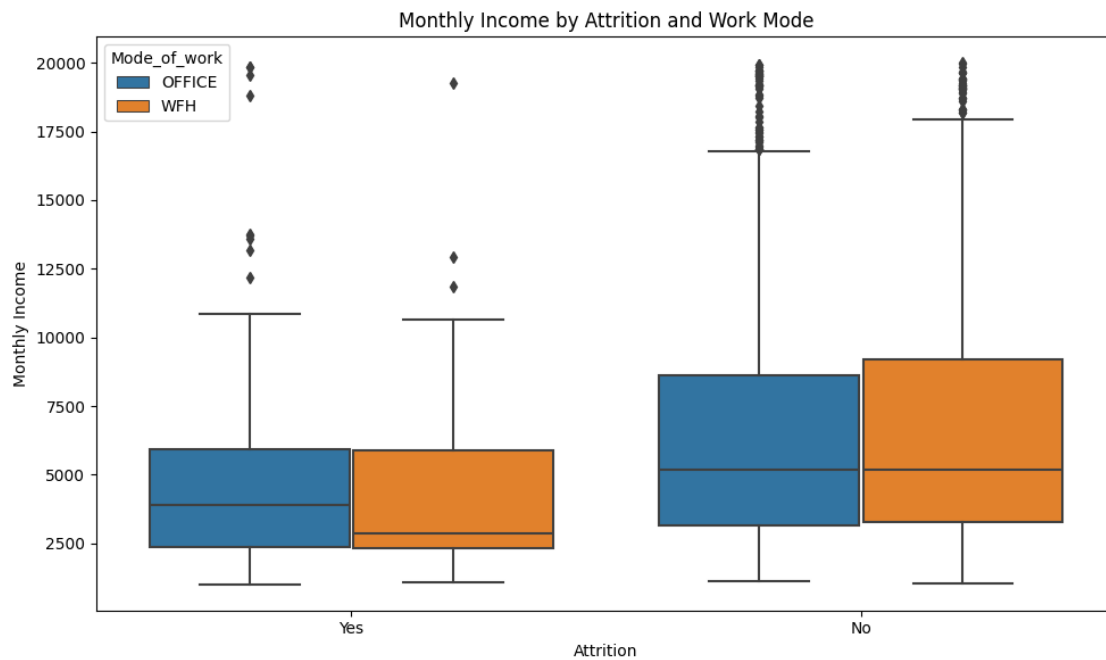
Summary: Younger employees are more likely to leave which reflects early career job hopping or better opportunities. Older employees seem more stable. Age seems to be a strong predictor of attrition.

```

[18]: # Monthly Income by Attrition and Work Mode
plt.figure(figsize=(10,6))
sns.boxplot(data=df, x='Attrition', y= 'MonthlyIncome', hue='Mode_of_work')
plt.title('Monthly Income by Attrition and Work Mode')
plt.xlabel('Attrition')
plt.ylabel('Monthly Income')

```

```
plt.tight_layout()
plt.show()
```



Summary: Lower income is associated with higher attrition whether the employee works remotely or in the office.

```
[21]: # Years at Company vs. Attrition
plt.figure(figsize=(10,6))
sns.violinplot(data=df, x='Attrition', y='YearsAtCompany', hue='Mode_of_work',
               split=True)
plt.title('Years at Company by Attrition and Work Mode')
plt.xlabel('Attrition')
plt.ylabel('Years at Company')
plt.tight_layout()
plt.show()
```



Summary: Short tenure is associated with higher attrition. This is a very strong predictor of attrition.

```
[26]: # Review column names
print(df.columns.tolist())

['Age', 'Attrition', 'BusinessTravel', 'Department', 'DistanceFromHome',
'Gender', 'JobInvolvement', 'JobLevel', 'JobRole', 'JobSatisfaction',
'MaritalStatus', 'MonthlyIncome', 'NumCompaniesWorked', 'OverTime',
'PercentSalaryHike', 'PerformanceRating', 'StockOptionLevel',
'TotalWorkingYears', 'TrainingTimesLastYear', 'YearsAtCompany',
'YearsSinceLastPromotion', 'YearsWithCurrManager', 'Higher_Education',
'Date_of_Hire', 'Date_of_termination', 'Status_of_leaving', 'Mode_of_work',
'Leaves', 'Absenteeism', 'Work_accident', 'Source_of_Hire', 'Job_mode']

[28]: # Drop unnecaessary features
df.drop(columns=['Date_of_Hire', 'Date_of_termination',
↳ 'Source_of_Hire', 'Status_of_leaving'],
        inplace=True)

[30]: # Remove Date_of_hire because already have YearsAtCompany
# Remove Date_of_termination because it is missing for active employees and
↳ could leak data
# Remove Status_of_leaving because it is only populated for those who left and
↳ could leak data
# Remove Source_of_hire because it is unlikely to be predictive
```

```
[32]: # Check missing values
print(df.isnull().sum())
```

```
Age                0
Attrition          0
BusinessTravel     0
Department        0
DistanceFromHome   0
Gender            0
JobInvolvement     0
JobLevel          0
JobRole           0
JobSatisfaction    0
MaritalStatus      0
MonthlyIncome      0
NumCompaniesWorked 0
OverTime          0
PercentSalaryHike  0
PerformanceRating  0
StockOptionLevel   0
TotalWorkingYears  0
TrainingTimesLastYear 0
YearsAtCompany     0
YearsSinceLastPromotion 0
YearsWithCurrManager 0
Higher_Education   0
Mode_of_work       0
Leaves            0
Absenteeism        0
Work_accident      0
Job_mode           0
dtype: int64
```

```
[34]: # Check MonthlyIncome distribution
print(df['Age'].value_counts())
```

```
Age
35    78
34    77
31    69
36    69
29    68
32    61
30    60
38    58
33    58
40    57
37    50
```

27	48
28	48
42	46
39	42
45	41
41	40
26	39
44	33
46	33
43	32
50	30
24	26
25	26
49	24
47	24
55	22
51	19
53	19
48	19
54	18
52	18
22	16
58	14
56	14
23	14
21	13
20	11
59	10
19	9
18	8
60	5
57	4

Name: count, dtype: int64

```
[36]: # Check MonthlyIncome distribution
print(df['MonthlyIncome'].value_counts())
```

MonthlyIncome	
2342	4
2559	3
5562	3
6347	3
3452	3
..	
3041	1
4707	1
3408	1
2218	1


```
10312      1
Name: count, Length: 1349, dtype: int64
```

```
[38]: # Encode target variable attrition into numbers 1 (yes) and 0 (no)
df['Attrition'] = df['Attrition'].str.strip().str.lower().map({'yes': 1, 'no': 0})
```

```
[40]: # Identify categorical features
categorical_cols = df.select_dtypes(include='object').columns.tolist()
categorical_cols
```

```
[40]: ['BusinessTravel',
      'Department',
      'Gender',
      'JobRole',
      'MaritalStatus',
      'OverTime',
      'Higher_Education',
      'Mode_of_work',
      'Work_accident',
      'Job_mode']
```

```
[42]: # Apply one hot encoding to turn categories into 0/1 columns
df = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
```

```
[44]: # Scale numeric columns to give similar range
df_original = df.copy()

from sklearn.preprocessing import StandardScaler

scaler=StandardScaler()
numeric_cols = ['DistanceFromHome', 'NumCompaniesWorked', 'PercentSalaryHike',
               'PerformanceRating',
               'StockOptionLevel', 'TotalWorkingYears',
               'TrainingTimesLastYear', 'YearsAtCompany',
               'YearsSinceLastPromotion', 'YearsWithCurrManager', 'Leaves',
               'Absenteeism']

df_scaled = df.copy()
df_scaled[numeric_cols] = scaler.fit_transform(df_scaled[numeric_cols])

comparison = pd.DataFrame({
    'Before Scaling (YearsAtCompany)': df_original['YearsAtCompany'],
    'After Scaling (YearsAtCompany)': df_scaled['YearsAtCompany']
})

print(comparison.head())
```

	Before Scaling (YearsAtCompany)	After Scaling (YearsAtCompany)
0	0	-1.144294
1	0	-1.144294
2	0	-1.144294
3	0	-1.144294
4	0	-1.144294

[46]: *# Review changes*

```
df.info()
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1470 entries, 0 to 1469
```

```
Data columns (total 41 columns):
```

#	Column	Non-Null Count	Dtype
0	Age	1470 non-null	int64
1	Attrition	1470 non-null	int64
2	DistanceFromHome	1470 non-null	int64
3	JobInvolvement	1470 non-null	int64
4	JobLevel	1470 non-null	int64
5	JobSatisfaction	1470 non-null	int64
6	MonthlyIncome	1470 non-null	int64
7	NumCompaniesWorked	1470 non-null	int64
8	PercentSalaryHike	1470 non-null	int64
9	PerformanceRating	1470 non-null	int64
10	StockOptionLevel	1470 non-null	int64
11	TotalWorkingYears	1470 non-null	int64
12	TrainingTimesLastYear	1470 non-null	int64
13	YearsAtCompany	1470 non-null	int64
14	YearsSinceLastPromotion	1470 non-null	int64
15	YearsWithCurrManager	1470 non-null	int64
16	Leaves	1470 non-null	int64
17	Absenteeism	1470 non-null	int64
18	BusinessTravel_Travel_Frequently	1470 non-null	bool
19	BusinessTravel_Travel_Rarely	1470 non-null	bool
20	Department_Research & Development	1470 non-null	bool
21	Department_Sales	1470 non-null	bool
22	Gender_Male	1470 non-null	bool
23	JobRole_Human Resources	1470 non-null	bool
24	JobRole_Laboratory Technician	1470 non-null	bool
25	JobRole_Manager	1470 non-null	bool
26	JobRole_Manufacturing Director	1470 non-null	bool
27	JobRole_Research Director	1470 non-null	bool
28	JobRole_Research Scientist	1470 non-null	bool
29	JobRole_Sales Executive	1470 non-null	bool
30	JobRole_Sales Representative	1470 non-null	bool
31	MaritalStatus_Married	1470 non-null	bool

32	MaritalStatus_Single	1470 non-null	bool
33	OverTime_Yes	1470 non-null	bool
34	Higher_Education_Graduation	1470 non-null	bool
35	Higher_Education_PHD	1470 non-null	bool
36	Higher_Education_Post-Graduation	1470 non-null	bool
37	Mode_of_work_WFH	1470 non-null	bool
38	Work_accident_Yes	1470 non-null	bool
39	Job_mode_FullTime	1470 non-null	bool
40	Job_mode_Part Time	1470 non-null	bool

dtypes: bool(23), int64(18)

memory usage: 239.9 KB

```
[46]:
```

	Age	Attrition	DistanceFromHome	JobInvolvement	JobLevel	\
0	37	1	2	2	1	
1	21	0	15	3	1	
2	45	0	6	3	3	
3	23	0	2	3	1	
4	22	0	15	3	1	

	JobSatisfaction	MonthlyIncome	NumCompaniesWorked	PercentSalaryHike	\
0	3	2090	6	15	
1	4	1232	1	14	
2	1	13245	4	14	
3	1	2322	3	13	
4	4	2871	1	15	

	PerformanceRating	...	MaritalStatus_Married	MaritalStatus_Single	\
0	3	...	False	True	
1	3	...	False	True	
2	3	...	True	False	
3	3	...	False	False	
4	3	...	False	True	

	OverTime_Yes	Higher_Education_Graduation	Higher_Education_PHD	\
0	True	True	False	
1	False	True	False	
2	True	False	False	
3	False	False	True	
4	False	False	True	

	Higher_Education_Post-Graduation	Mode_of_work_WFH	Work_accident_Yes	\
0	False	False	False	
1	False	True	False	
2	True	True	False	
3	False	False	True	
4	False	True	False	

	Job_mode_FullTime	Job_mode_Part Time
0	False	False
1	False	True
2	False	False
3	True	False
4	False	False

[5 rows x 41 columns]

```
[48]: # Check Class Imbalance
df['Attrition'].value_counts(normalize=True)
```

```
[48]: Attrition
0    0.838776
1    0.161224
Name: proportion, dtype: float64
```

```
[50]: # Attrition is imbalanced with only 16% positive. I will address this in future
      ↪ milestones.
```

```
[60]: # Import necessary libraries
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix,
      ↪ roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns
from xgboost import XGBClassifier
```

```
[62]: # I focused only on remote workers from cleaned and scaled dataset
df_remote = df_scaled[df_scaled['Mode_of_work_WFH'] ==1]
```

```
[64]: # Define X and y
# Drop target variable from features
X = df_remote.drop('Attrition', axis=1)
y = df_remote['Attrition']
```

```
[66]: # Train/Test Split (Stratify to preserve class imbalance)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
      ↪ stratify=y, random_state=42)
```

```
[68]: # Train logistic regression (baseline model)
# Using class_weight='balanced' to account for class imbalance
# Logistic Regression was used as a baseline model due to interpretability and
      ↪ speed
lr = LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)
```

```
lr.fit(X_train, y_train)
```

```
[68]: LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)
```

```
[70]: # Prediction and predicted probabilities
y_pred_lr = lr.predict(X_test)
y_prob_lr = lr.predict_proba(X_test)[: , 1]
```

```
[72]: # Evaluate logistic regression
print('Logistic Regression Results:\n')
print(classification_report(y_test,y_pred_lr))
print('Confusion Matrix:')
print(confusion_matrix(y_test,y_pred_lr))
print('ROC AUC Score:', roc_auc_score(y_test, y_prob_lr))
```

Logistic Regression Results:

	precision	recall	f1-score	support
0	0.96	0.73	0.83	130
1	0.36	0.83	0.51	24
accuracy			0.75	154
macro avg	0.66	0.78	0.67	154
weighted avg	0.87	0.75	0.78	154

Confusion Matrix:

```
[[95 35]
 [ 4 20]]
```

ROC AUC Score: 0.8737179487179487

```
[74]: # Train Random Forest
# Random Forest was used because it handles non-linear relationships and
# performs well with class weights
rf = RandomForestClassifier(class_weight='balanced', n_estimators=100,
# random_state=42)
rf.fit(X_train, y_train)
```

```
[74]: RandomForestClassifier(class_weight='balanced', random_state=42)
```

```
[76]: # Predictions
y_pred_rf = rf.predict(X_test)
y_prob_rf = rf.predict_proba(X_test)[: , 1]
```

```
[78]: # Evaluate Random Forest
print('Random Forest Results:\n')
print(classification_report(y_test,y_pred_rf))
print('Confusion Matrix:')

```

```
print(confusion_matrix(y_test,y_pred_lr))
print('ROC AUC Score:', roc_auc_score(y_test, y_prob_rf))
```

Random Forest Results:

	precision	recall	f1-score	support
0	0.85	1.00	0.92	130
1	1.00	0.04	0.08	24
accuracy			0.85	154
macro avg	0.92	0.52	0.50	154
weighted avg	0.87	0.85	0.79	154

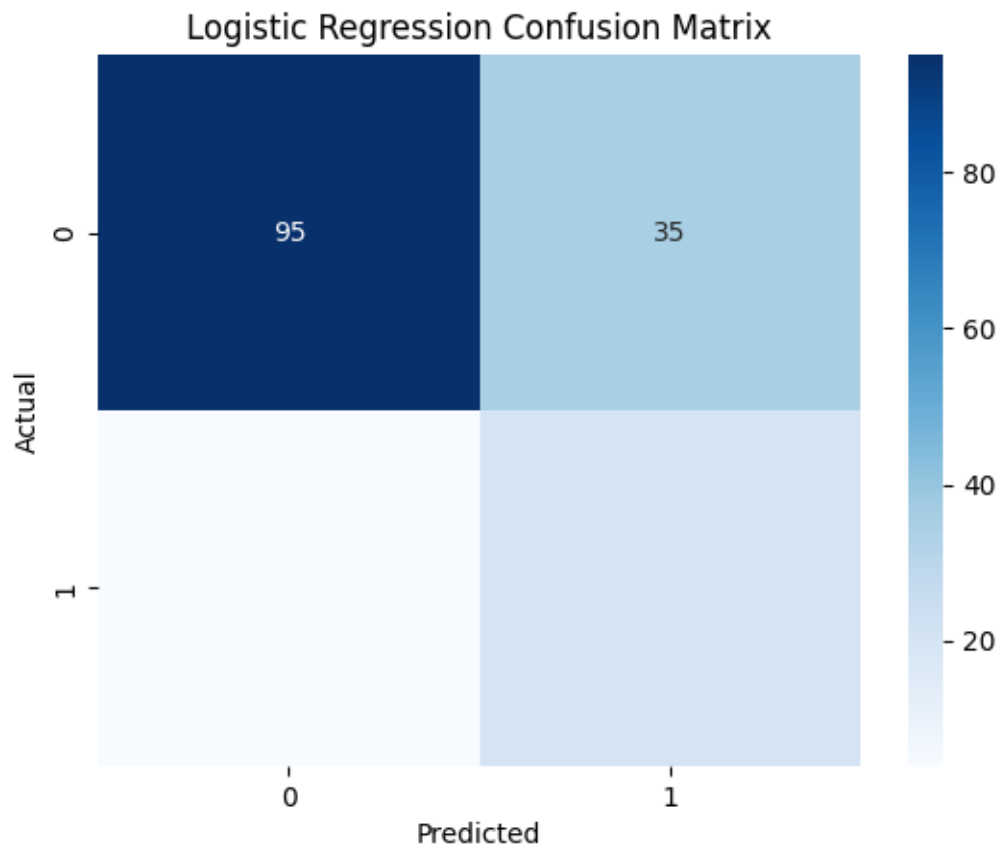
Confusion Matrix:

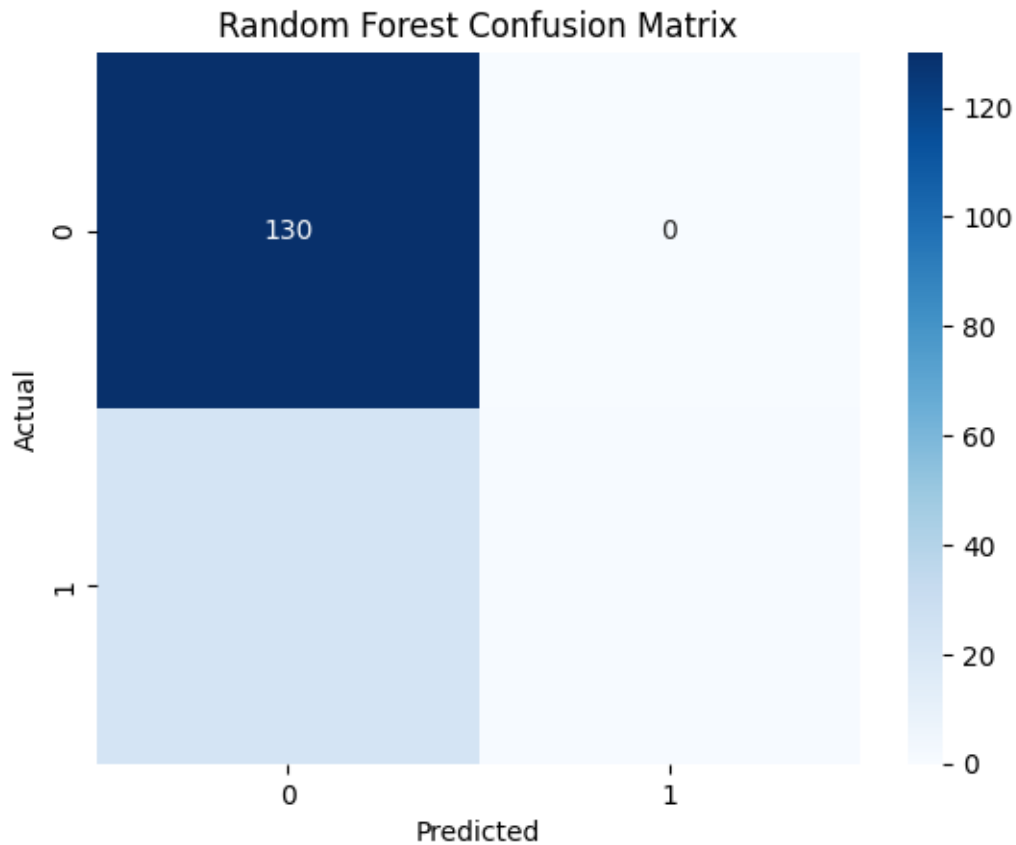
```
[[95 35]
 [ 4 20]]
```

ROC AUC Score: 0.8020833333333334

```
[82]: # Visualization of Confusion Matrix
def plot_conf_matrix(y_true, y_pred, title):
    cm = confusion_matrix(y_true, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.title(title)
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.show()

plot_conf_matrix(y_test, y_pred_lr, 'Logistic Regression Confusion Matrix')
plot_conf_matrix(y_test, y_pred_rf, 'Random Forest Confusion Matrix')
```





```
[107]: xgb = XGBClassifier(  
        n_estimators=500,  
        learning_rate=0.05,  
        max_depth=4,  
        subsample=0.8,  
        colsample_bytree=0.8,  
        reg_lambda=1.0,  
        objective='binary:logistic',  
        eval_metric='auc',  
        scale_pos_weight=scale_pos_weight,  
        random_state=42,  
        n_jobs=-1  
    )
```

```
[80]: # Handle class imbalance with XGBoost  
pos = y_train.sum()  
neg = len(y_train) - pos  
scale_pos_weight = (neg / pos) if pos > 0 else 1.0
```



```
[85]: X_train_xgb = X_train.astype('float32')
      X_test_xgb = X_test.astype('float32')
```

```
[109]: # Early stopping to prevent overfitting
      fitted = False
      try:
          from xgboost.callback import EarlyStopping
          es = [EarlyStopping(rounds=30, metric_name = 'auc', save_best=True)]
          xgb.fit(X_train_xgb, y_train, eval_set=[(test_xgb, y_test)], callbacks = es, verbose = False)
          fitted = True
      except Exception:
          try:
              xgb.fit(X_train_xgb, y_train, eval_set[(X_test_xgb, y_test)], early_stopping_rounds=30, verbose=False)
              fitted = True
          except Exception:
              xgb.fit(X_train_xgb, y_train, verbose=False)
```

```
[111]: # Predictions & Metrics
      y_prob_xgb = xgb.predict_proba(X_test_xgb)[: , 1]
      y_pred_xgb = (y_prob_xgb >=0.5).astype(int)
```

```
[115]: print('XGBoost Results:\n')
      print(classification_report(y_test, y_pred_xgb))
      print('Confusion Matrix:')
      print(confusion_matrix(y_test, y_pred_xgb))
      print('ROC AUC Score:', roc_auc_score(y_test, y_prob_xgb))
      plot_conf_matrix(y_test, y_pred_xgb, 'XGBoost Confusion Matrix')
```

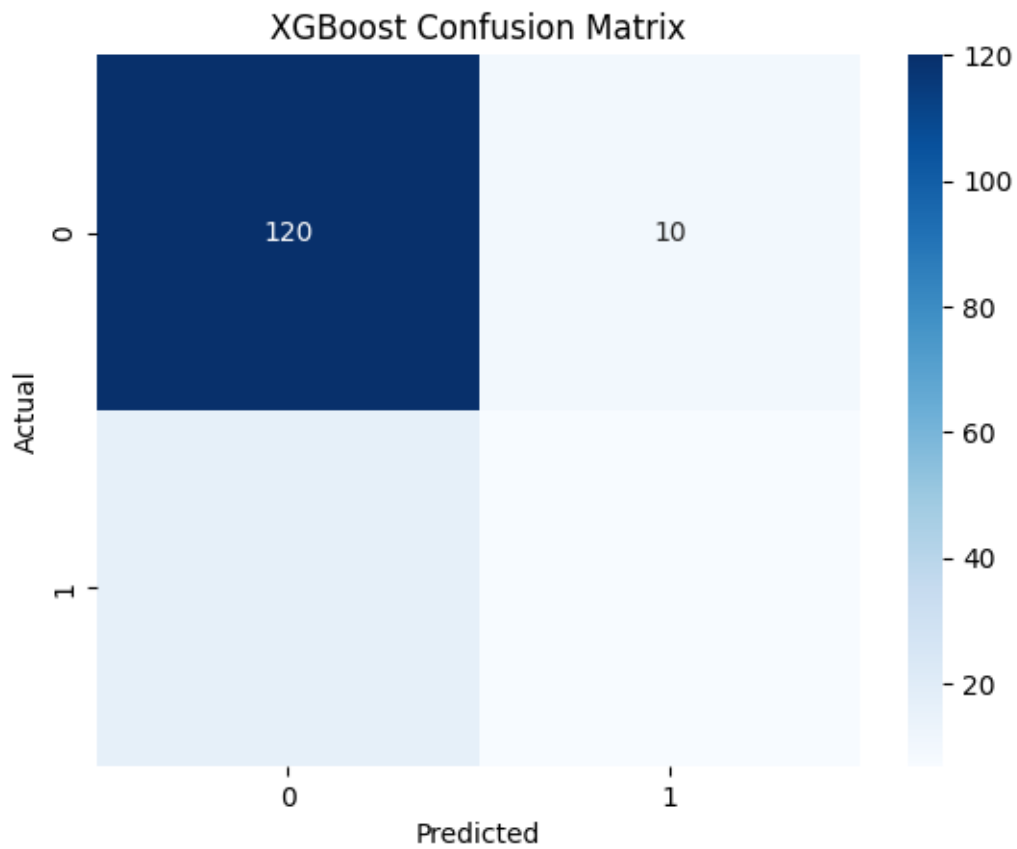
XGBoost Results:

	precision	recall	f1-score	support
0	0.88	0.92	0.90	130
1	0.41	0.29	0.34	24
accuracy			0.82	154
macro avg	0.64	0.61	0.62	154
weighted avg	0.80	0.82	0.81	154

Confusion Matrix:

```
[[120 10]
 [ 17  7]]
```

ROC AUC Score: 0.7887820512820513



[]: